

花卉分类器 · 训练过程记录

本文记录三次真实训练的完整配置、逐轮数据、曲线与结论。

数据来源： docs/logs/history_Adam.json 、 docs/logs/history_SGD.json （由 train.py 自动落盘） ，

以及实验 0 的终端输出（当时落盘代码尚未加入）。

用途：既是提交材料，也是"以后怎么调参"的证据链 —— 想重画曲线不必重训。

一、实验总览

三次训练固定随机种子 42，因此初始权重完全相同，唯一变量是优化器与学习率。

#	配置	末轮 train acc	末轮 val acc	最佳 val acc	结论
0	SGD lr=1e-4, momentum=0.9	0.2973	0.2939	0.3186	✗ 完全没学到
1	Adam lr=1e-4	0.7687	0.6749	0.6946 (epoch 12)	✓ 最优
2	SGD lr=1e-2, momentum=0.9	0.7023	0.6732	0.6732 (epoch 15)	✓ 收敛，略逊

最终提交模型：实验 1（Adam），test 集准确率 71.28%。

一句话结论：调参的优先级不是"选哪个优化器更优"，而是先确认超参数之间是否配对。

配对错误带来的差距（约 38 个百分点）远大于优化器之间本身的差距（约 2 个百分点）。

二、统一训练配置

项	取值
数据集	train 2425 / val 609（每类各 20%，分层）
batch_size	32
epochs	15
损失函数	nn.CrossEntropyLoss()
随机种子	42
num_workers	2
计算设备	CUDA（RTX 5060 Laptop）
单轮耗时	约 70 s（总计约 17 min / 组）

训练循环五步（顺序不可换）：

```
for inputs, labels in loader:
    optimizer.zero_grad()           # ① 清梯度（backward 是累加，不是赋值）
    outputs = model(inputs)         # ② 前向
    loss = criterion(outputs, labels) # ③ 算损失
    loss.backward()                 # ④ 反向传播
    optimizer.step()                # ⑤ 更新参数
```

两处开关：训练前 `model.train()`，评估前 `model.eval()`。

`eval()` 漏掉的后果：验证时 Dropout 仍在随机丢弃，

`if val_acc > best_val_acc` 保存下来的会是“**运气最好的一轮**”而非真正最优模型。

三、实验 0：SGD $lr=1e-4$ —— 一次「模型不学习」的完整诊断

3.1 现象

```
epoch 1/15  train loss 1.6099  acc 0.1769 | val loss 1.6098  acc 0.1823
epoch 15/15 train loss 1.6010  acc 0.2973 | val loss 1.6007  acc 0.2939
```

3.2 两条关键证据

证据 A：loss 几乎没动。

15 轮 1140 个训练步，loss 只从 1.6099 降到 1.6010（降幅 0.0089）。

而 $\ln(5) = 1.6094$ 正是“五分类瞎猜”的损失值——

说明模型输出几乎仍是“五个类各 20%”的均匀分布，**参数压根没被推动过**。

证据 B：train acc $\approx$ val acc。

0.2973 vs 0.2939，两个数几乎一样。

这一步的判读方法比结果更重要：

- 若 train 高、val 低 → **过拟合**（模型记住了训练集，没学到规律）；
- 若 train 和 val 都低（本例）→ **欠拟合**（模型连训练集都记不住）。

本例是欠拟合，所以**问题不在数据、不在网络结构**，只能出在“参数有没有被推动”上。

3.3 根因：学习率与优化器不配对

优化器	怎么决定每步走多远	常用 lr
Adam	按每个参数的梯度历史 自动缩放 步长，所以能吃很小的 lr	1e-4 ~ 3e-3
SGD	完全按给定的 lr 走， 没有任何自适应	1e-2 ~ 1e-1 (配 momentum)

lr=1e-4 是给 Adam 的量级；配给 SGD 相当于"开着一辆没有油门辅助的车，却只踩 1% 的油门"——差的不是 10%，是 100 倍。

3.4 修法与验证

两条路（见实验 1 / 实验 2）：

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4) # 路线 A
optimizer = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9) # 路线 B
```

两条都跑通了，验证了根因确实是 lr 而非"SGD 不行"。

四、实验 1: Adam lr=1e-4 (最终提交模型)

4.1 逐轮数据

epoch	train loss	train acc	val loss	val acc
1	1.4631	0.3361	1.3233	0.3974
2	1.2574	0.4482	1.3128	0.4269
3	1.1559	0.5155	1.1157	0.5271
4	1.0609	0.5571	1.1604	0.4975
5	1.0221	0.5876	1.0329	0.6043
6	0.9291	0.6375	0.9591	0.6453
7	0.8750	0.6487	1.0417	0.5961
8	0.8561	0.6565	0.8917	0.6700
9	0.8063	0.6903	0.9793	0.6683
10	0.7608	0.7060	0.9996	0.6585
11	0.7072	0.7303	0.8855	0.6749
12	0.6830	0.7460	0.8417	0.6946 ← best
13	0.6414	0.7604	0.9619	0.6535
14	0.6242	0.7612	0.9303	0.6814
15	0.6310	0.7687	0.9394	0.6749

4.2 观察

1. **收敛正常**: train acc 从 0.34 一路爬到 0.77, 没有任何"卡住"的迹象;
2. **最佳出现在 epoch 12** (val 0.6946), 此后 val 在 0.65~0.68 之间波动 →
best_model.pth 保存的正是这一轮, 说明"按 val 选最优"的机制在正常工作;
3. **过拟合起点约在 epoch 8**: val_loss 从第 8 轮起在 0.88~1.00 横盘甚至回升,
而 train_loss 继续降到 0.63 → 模型开始"记"训练集了;
4. **train / val 差距逐轮扩大** (末轮 0.094) → 轻度过拟合。Dropout(0.5) 已在抑制,
但 5800 万参数配 2425 张图, 这个趋势是预期的;
5. **val 到末轮仍未见顶** → EPOCHS=15 偏少, 值得试 25 轮。

4.3 产物

- 最优权重: checkpoints/best_model.pth (233 MB, 58,301,829 × 4 字节)

- 曲线图: docs/figures/fig_curve_Adam.png
- 训练历史: docs/logs/history_Adam.json

五、实验 2: SGD $\text{lr}=1\text{e-}2$, momentum=0.9

5.1 逐轮数据

epoch	train loss	train acc	val loss	val acc
1	1.5930	0.2726	1.5193	0.3563
2	1.4095	0.3876	1.3423	0.4647
3	1.2778	0.4491	1.2525	0.4647
4	1.2210	0.4565	1.1829	0.4828
5	1.1247	0.5340	1.1772	0.5090
6	1.0901	0.5625	1.0778	0.5550
7	1.0474	0.5757	1.1190	0.5484
8	1.0003	0.6148	0.9777	0.6453
9	0.9147	0.6470	1.0566	0.6059
10	0.9520	0.6272	1.0329	0.6289
11	0.8845	0.6623	0.9482	0.6585
12	0.8167	0.6920	0.9320	0.6371
13	0.7995	0.6845	0.9654	0.6371
14	0.7948	0.6866	0.9124	0.6716
15	0.7807	0.7023	0.8792	0.6732 ← best

5.2 观察

1. 把 lr 提到 $1\text{e-}2$ 后, SGD 立刻活了过来: val acc 从实验 0 的 0.29 跳到 0.67 ——
同一份数据、同一个网络、同一个种子, 只改了 lr ;
2. 上升比 Adam 慢但更稳: SGD 的 val 曲线抖动更小, 末轮还在爬 (0.6732 就是最佳);
3. 末轮 loss 仍在下降 (train 0.78 / val 0.88) → 若给更多轮次, SGD 有可能追平甚至反超 Adam;
4. 训练集 acc (0.7023) 低于 Adam (0.7687) → SGD 在小数据量上“记得更少”,
这正是它泛化有时更好的原因, 但在 15 轮的预算内还没体现出来。

5.3 产物

- 曲线图: docs/figures/fig_curve_SGD.png
- 训练历史: docs/logs/history_SGD.json

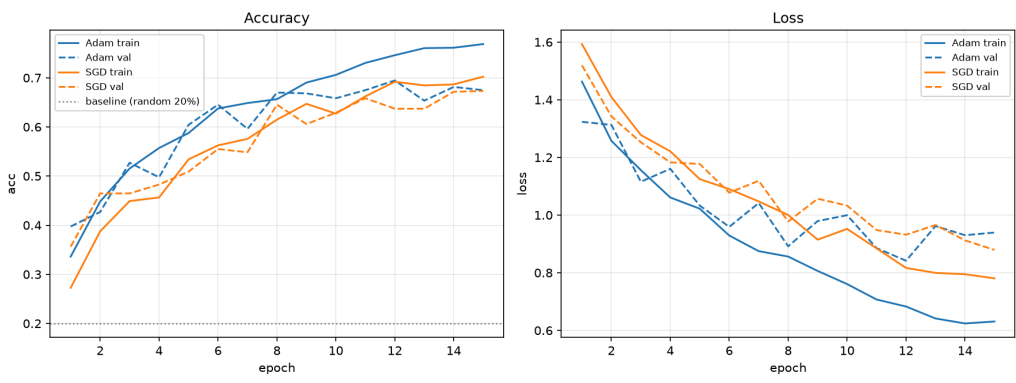
⚠ 注意: checkpoints/best_model.pth 文件名是固定的, 实验 2 覆盖了实验 1 的权重。

实验 1 (Adam) 的训练历史完整保留在 json 中, 需要其权重时重跑一次 train.py 即可。

这是一个已记录的已知问题 (见提交报告 7.5), 改进方向是把 checkpoint 也按优化器命名。

六、三组对照的完整结论

6.1 对照曲线



左: 准确率 (实线 train / 虚线 val, 灰点线为 20% 随机基线) ; 右: 损失。

该图由 docs/labs/compare_history.py 从两个 json 直接重画, 不需要重训。

6.2 结论

问题	答案
哪个优化器更好?	15 轮内 Adam 略优 (0.6946 vs 0.6732), 差距约 2 个百分点
最大的一次提升来自哪里?	不是换优化器, 而是把 lr 配对 (SGD 从 0.29 → 0.67, 涨了 38 个百分点)
为什么实验 0 完全学不动?	SGD + lr=1e-4 差 100 倍, 每一步的更新量小到几乎为零
调参应该先做什么?	先检查超参数之间的"适配关系" (优化器 → 学习率), 再比较谁更优

6.3 后续可做的实验 (尚未做)

1. **Adam @ 25 epochs** —— val 到末轮仍上升, 加轮次大概率还能涨;
2. **SGD @ 25 epochs** —— 同理, 且 SGD 曲线更稳, 长跑表现值得看;
3. **SGD + 学习率衰减** —— 例: 每 10 轮 ×0.1, 兼顾前期的快和后期的稳;

4. Adam lr 扫描 ($3e-4$ / $1e-3$) —— 确认 $1e-4$ 是否就是最优点。

七、训练相关的踩坑记录

7.1 验证集准确率偏低 $\neq$ 验证集的问题

误判: "为什么验证集准确率这么低?"

实情: train acc 0.2973 $\approx$ val acc 0.2939 —— 两边都低, 是欠拟合, 不是验证集被"区别对待"。

判读口诀: 先看 train 和 val 差多少, 再判断是过拟合还是欠拟合。

7.2 曲线图右边空白 —— 「陈图」陷阱

现象: 训练完成后生成的曲线图, Accuracy 子图有条线, Loss 子图整块空白。

代码复查并模拟运行正确 (两个子图各 2 条线)。

根因 (时间戳钉死):

evaluate.py 保存时间	22:40	
训练进程启动时间	~22:37	← 早了 3 分钟
曲线图生成时间	22:55	

Python 启动时只 import 一次模块 —— 进程跑起来之后再改文件, 正在运行的进程毫不知情, 用的还是 import 那一刻的旧版本。

规律: 改完代码必须重启进程才生效。

加固: train.py 现在把训练历史落盘为 json, 并提供 compare_history.py 从 json 重画 —— 以后重画曲线不必重训, 也就不会再撞这个坑。

7.3 训练耗时异常的两轮

epoch 11 用了 684.9s、epoch 14 用了 1993.2s (其余约 70s)。

原因是那段时间电脑休眠/挂起, 而 time.time() 把挂起时间也算进了"这一轮耗时"。

不影响训练结果, 只是计时被污染。

7.4 解释器选错

截图里出现 C:\Users\Henry\AppData\Roaming\uv\python\...\python.exe ——

不是项目的虚拟环境, 那里没装 torch, 会报 ModuleNotFoundError: No module named 'torch', 极易被误判成"环境坏了"。

规矩: 本项目永远用全路径 D:\ML\.venv_gpu\Scripts\python.exe, 不要裸敲 python。

八、复现方式

```
cd /d D:\ML.2

# ① 数据管道（若 processed/ 已存在可跳过）
D:\ML\.venv_gpu\Scripts\python.exe src\data\clean.py
D:\ML\.venv_gpu\Scripts\python.exe src\data\split.py
D:\ML\.venv_gpu\Scripts\python.exe src\data\dataset.py      # 自检

# ② 训练（把 src\train.py 里的 optimizer 换成目标配置）
D:\ML\.venv_gpu\Scripts\python.exe src\train.py

# ③ 评估 test 集
D:\ML\.venv_gpu\Scripts\python.exe src\evaluate.py

# ④ 重画多组对照曲线（读 docs/logs/*.json，不必重训）
D:\ML\.venv_gpu\Scripts\python.exe docs\labs\compare_history.py
```

每组训练会自动产生：

- docs/logs/history_<优化器名>.json —— 逐轮 loss / acc
- docs/figures/fig_curve_<优化器名>.png —— 该组曲线
- checkpoints/best_model.pth —— 最优权重（会被下一组覆盖，注意）